

Methodological Audit of Machine Learning Notebooks

Audit Engine AI

1 Context

To audit the methodology of Machine Learning before MLOps, there is no turnkey commercial agent, but rather a deterministic normative workflow based on scientific rigor and bias prevention.

2 ML Methodological Audit Normative

The AI engine must evaluate the code by searching for anti-methodological patterns divided into three main axes:

2.1 Data Leakage — Critical Severity

- **Disordered Fit/Transform:** `StandardScaler`, `MinMaxScaler`, or other transformers apply `.fit()` or `.fit_transform()` to the entire dataset before executing `train_test_split`.
- **Target Leakage:** Feature creation computing global statistical aggregations that involve the target variable (`y`).
- **Viciated Cross-Validation:** Application of dimensionality reduction (e.g., PCA) or global scaling before cross-validation, instead of inside a scikit-learn Pipeline.

2.2 Reproducibility and Data Splitting

- **Randomness Control:** Absence of global or local seeds (`random_state`, `seed`) in data splits, clustering algorithms, or stochastic models.
- **Wrong Split Strategy:** Use of standard random splitting (`random_split`) on sequential or temporal data (requires Time-Series Split) or on severely imbalanced datasets (requires stratified splitting with `stratify`).

2.3 Modeling and Metrics Coherence

- **Inadequate Metric:** Evaluation based on Accuracy for imbalanced class problems instead of F1-Score, PR-AUC, or ROC-AUC.
- **Missing Baseline:** Training the final model without prior comparison against a naive or reference model (dummy/baseline classifier).

3 Orchestration Flow for the Audit Engine AI

3.1 Context Extraction and AST (Static Syntax)

Before passing the code to the LLM, a script extracts the Data Flow Graph from the AST of the normalized code. This locates which cells declare dataframes, transformations, and where the first `.fit()` occurs.

3.2 Specialized Sub-Agent Evaluation

- **Leakage and Transformations Agent:** Analyzes solely the temporal sequence of data flow between DataFrames to detect Data Leakage.
- **Metrics and Validation Agent:** Analyzes the model arguments and metrics called in the evaluation subset.

3.3 Structured Consolidation

The engine combines failures and generates a deterministic JSON response to automate corrections.

4 Recommended Agent Output Schema

```
1 {  
2   "cell_index": 5,  
3   "severity": "CRITICAL",  
4   "category": "Data_Leakage",  
5   "issue": "StandardScaler.fit_transform()_executed  
6   before_train_test_split()",  
7   "line_code": "X_scaled=_scaler.fit_transform(X)",  
8   "suggested_fix": "X_train,_X_test=_train_test_split(X,...)\n  
9   X_train_scaled=_scaler.fit_transform(X_train)\n10  X_test_scaled=_scaler.transform(X_test)"  
11 }
```

5 Audit Workflow Diagram

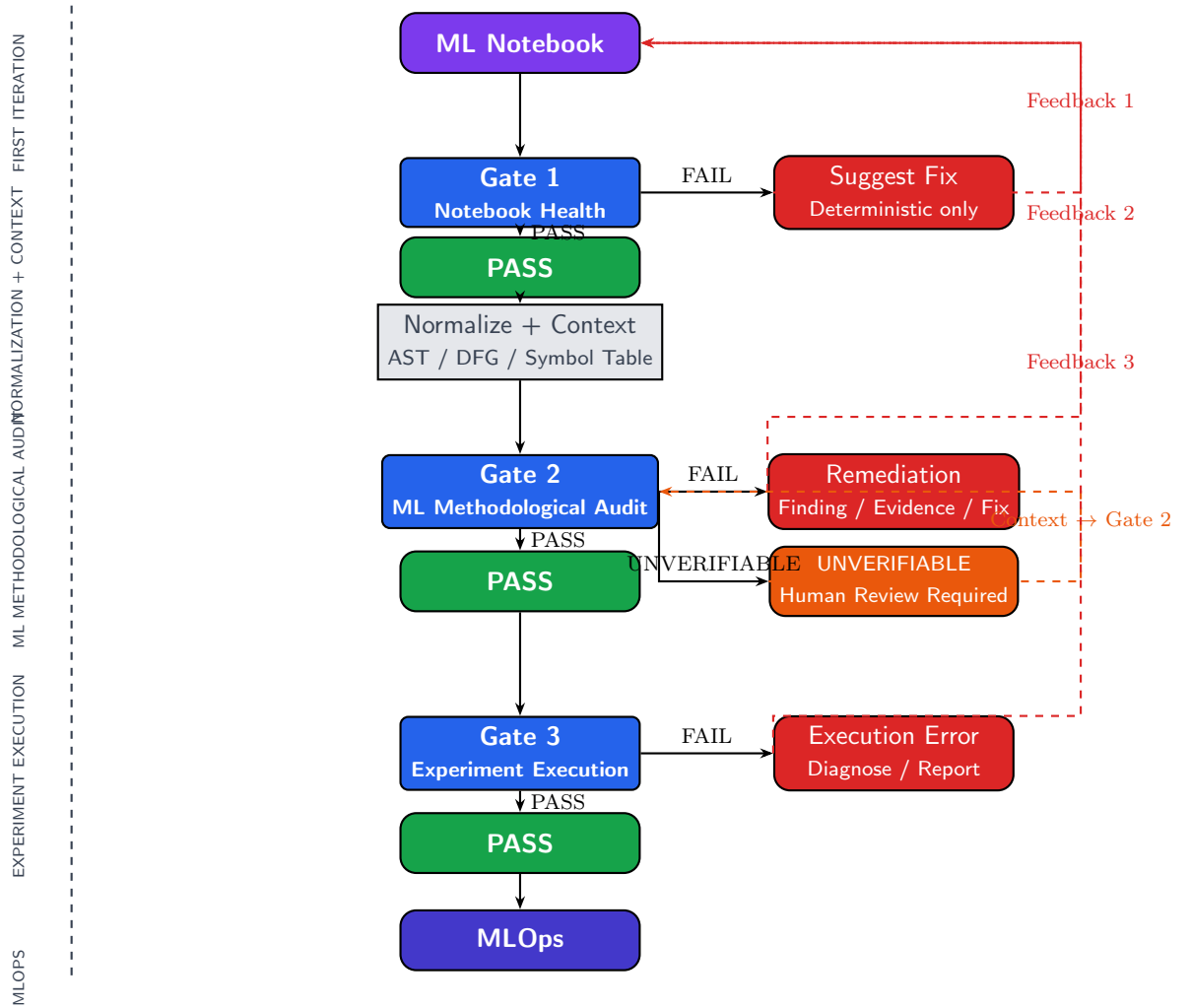


Figure 1: Audit Workflow — Three-gate deterministic methodology for ML notebook validation prior to MLOps execution.

References

- Yang, C., Brower-Sinning, R., Lewis, G., and Kästner, C. Data leakage in notebooks: Static detection and better processes. In *Proc. 37th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE '22)*, 2022. <https://doi.org/10.1145/3551349.3556918>
- Subotić, P., Bojanić, U., and Stojić, M. Statically detecting data leakages in data science code. In *Proc. 11th ACM SIGPLAN Int. Workshop on the State Of the Art in Program Analysis (SOAP '22)*, 2022. <https://doi.org/10.1145/3520313.3534657>
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. on Knowledge Discovery from Data*, 6(4):15:1–15:21, 2012. <https://doi.org/10.1145/2382577.2382579>
- Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. The ML test score: A rubric for ML production readiness and technical debt reduction. In *Proc. 2017 IEEE Int. Conf. on Big*

Data (Big Data '2017), pages 1123–1132, 2017. <https://doi.org/10.1109/BigData.2017.8258038>

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pages 2503–2511, 2015. <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>

National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, January 2023. NIST AI 100-1. <https://doi.org/10.6028/NIST.AI.100-1>